

Relatório de Auditoria de Segurança

Folium Print (Desktop & Web App)

Revisão defensiva exaustiva orientada a vulnerabilidades de injeção (DOM-XSS), controle de privilégios nativos, persistência local e gestão de segredos.

DATA DE EMISSÃO**30 de Setembro de 2026****AUDITOR RESPONSÁVEL****Antigravity AI Security Team****ESCOPO AVALIADO****100% do Código (Frontend React + Tauri v2 Rust)****STACK TECNOLÓGICA****React 18 / Vite 6 / Tailwind 3.4 / Tauri v2 / Rust 2024****Nota Metodológica & Adaptação da Stack:**

O *Folium Print* é uma aplicação desktop/web focada em **arquitetura 100% offline e local-first**, sem banco de dados remoto (Supabase/PostgreSQL), sem servidor backend centralizado e sem endpoints REST/GraphQL multi-inquilino. As categorias foram mapeadas para essa realidade:

- Banco sem Tranca:** Não há RLS ou banco remoto; os dados residem isolados no `localStorage` do navegador/webview. Verificada a falta de integridade criptográfica local.
- Permissão no Navegador:** Inexistência de RBAC multi-usuário; o equivalente são as permissões IPC do Tauri (`capabilities/default.json`). Verificado que apenas `core:default` está ativo.
- IDOR:** Sem rotas HTTP remotas; avaliada a colisão de IDs na importação de arquivos locais (`.folium.json`).
- Chaves Expostas:** Varredura de 100% dos 11 commits do histórico git, configs e bundle final (Resultado: Limpo / Ponto Forte).
- Inputs sem Tratamento (XSS):** Principal vetor de risco (sanitização de SVG com regex, interpolação em exportação HTML, `doc.write` no spooler e CSP do Tauri).

Resumo Executivo

A auditoria identificou **5 achados técnicos**: **3 de Alta Severidade** (falhas no processamento e higienização de SVG/HTML com risco de XSS), **1 de Média Severidade** (CSP nativo nulo no Tauri e unsafe-inline) e **1 de Baixa Severidade** (sanitização incompleta de PII na exportação).

5

TOTAL ACHADOS

0

CRÍTICA

3

ALTA

1

MÉDIA

4

PONTOS FORTES



Pontos Fortes (Protegido com Evidência)

- ✓ **Zero Chaves Hardcoded:** Varredura exaustiva dos 11 commits do histórico git e do bundle final confirmou ausência total de tokens ou API keys privadas.
- ✓ **Operação 100% Offline:** Nenhuma requisição externa (sem fetch/telemetria oculta); processamento gráfico estritamente confinado ao host local.
- ✓ **Sandbox Estrito no Tauri v2:** O arquivo capabilities/default.json restringe o runtime a core:default, sem expor APIs perigosas do sistema operacional.
- ✓ **Validação Rígida em Barcode/QR:** Validação de dígitos em CodeRenderer.tsx impede injeção ou falhas por valores de entrada corrompidos.

Pontos Fracos (Riscos Centrais)

- ✗ **Regex Bypass em Sanitizador SVG:** sanitizeSvg.ts permite evasão por tags aninhadas, attributes sem espaço e codificação HTML de URLs.
- ✗ **XSS na Exportação HTML:** templateFileIO.ts interpola dados de modelos sem escapar entidades HTML, gerando arquivos exportados vulneráveis.
- ✗ **doc.write em Iframe de Impressão:** printService.ts escreve markup direto no iframe, abrindo brecha para manipulação de localStorage.
- ✗ **CSP Desabilitado no Tauri:** tauri.conf.json define "csp": null e index.html permite 'unsafe-inline'.

Tabela Geral de Achados

SEVERIDADE	ARQUIVO & LINHA	DESCRIÇÃO RESUMIDA DA VULNERABILIDADE
ALTA	src/utils/sanitizeSvg.ts:31-45	Bypass de sanitização SVG por expressões regulares (DOM-based XSS via SVG).
ALTA	src/utils/templateFileIO.ts:50-127	Injeção de HTML/XSS armazenado na exportação de templates para arquivo HTML autônomo.
ALTA	src/utils/printService.ts:68-140	Injeção de código via doc.write em iframe de impressão sem sandbox ou isolamento de origem.
MÉDIA	src-tauri/tauri.conf.json:23	CSP nativo desabilitado ("csp": null) e uso permissivo de 'unsafe-inline' no frontend.
BAIXA	src/utils/sanitization.ts:44-67	Higienização de dados pessoais (LGPD) incompleta em exportações de modelos compartilháveis.

Detalhamento dos Achados

1. Bypass de Sanitização SVG e DOM-XSS via Injeção de Vetores

Arquivo: src/utils/sanitizeSvg.ts:31-45

Consumidores: src/utils/dynamicRenderer.tsx:82, 186 | src/components/VisualTemplateEditorModal.tsx:730, 812

Explorabilidade: Importação de arquivo vetorial .svg ou template .folium manipulado por terceiros.

```
31: for (const tag of DANGEROUS_TAGS) {
32:   const tagRegex = new RegExp(`<${tag}[^>]*>[\s\S]*?</${tag}>`, 'gi');
33:   clean = clean.replace(tagRegex, '');
...
39: clean = clean.replace(/\s+on[a-z]+=(("["]|'')*["']|[\s>])/gi, '');
42: clean = clean.replace(/(href|xlink:href)=("["]|'')*\s*javascript:("["]|'')*/gi, '$1="")');
```

Por que é explorável: A higienização de marcação SVG com Regex é vulnerável a evasões conhecidas: (1) tags aninhadas como <scr<script>ipt> onde apenas o par interno é removido; (2) manipuladores de eventos delimitados por barra sem espaço prévio (ex: <svg/onload=alert(1)>); e (3) entidades codificadas em URLs (ex: href="javascript:..."). Ao injetar via dangerouslySetInnerHTML, scripts arbitrários são disparados.

ALTA

2. Injeção de Código HTML na Geração de Arquivo Imprimível Exportado

Arquivo: src/utils/templateFileIO.ts:50-70, 76, 124-127

Função: generatePrintableHtml(template) / exportTemplateAsFile(..., 'html')

Explorabilidade: Usuário exporta modelo com payload malicioso no nome ou valores dos campos e o abre no navegador.

```
50: const content = field.defaultValue || field.label || '';
52: return `<div style="..."><span>${content}</span></div>`;
...
76: <title>${template.name} - Folium Print</title>
124: ${template.backgroundSvg ? `<div class="background-layer">${template.backgroundSvg}</div>` : ''}
```

Por que é explorável: As variáveis template.name, content e template.backgroundSvg são interpoladas diretamente em literais de string sem nenhuma conversão de caracteres para entidades HTML (< ; >). Além disso, backgroundSvg sequer passa por sanitização antes de ser impresso no arquivo HTML exportado, executando scripts maliciosos quando aberto no browser.

ALTA

3. Injeção de Código via doc.write em Iframe de Impressão Física

ALTA

Arquivo:src/utils/printService.ts:68-76, 139-141

Função:executePixelPerfectPrint(...)

68: doc.write(`<!DOCTYPE html>...<title>Folium Print - \${template.name}</title>...\${htmlContent}`);

Por que é explorável: O serviço de impressão instancia um <iframe> sem restrições de sandbox e injeta o documento com doc.write. Quebrar a tag de título com </title><script> permite execução imediata com acesso total a window.parent.localStorage, podendo apagar, alterar ou exfiltrar todos os modelos e configurações salvos.

4. Ausência de CSP Nativo no Tauri v2 e 'unsafe-inline' no Frontend

MÉDIA

Arquivos:src-auri/auri.conf.json:22-24 | index.html:9

Por que é explorável: Desativar o CSP no Tauri com "csp": null elimina a defesa em profundidade nativa. Paralelamente, admitir 'unsafe-inline' na tag meta do HTML autoriza a execução imediata de qualquer script injetado nas falhas 1, 2 e 3, neutralizando a proteção do navegador contra XSS.

5. Sanitização de PII Incompleta na Exportação de Modelos (LGPD)

BAIXA

Arquivos:src/utils/sanitization.ts:44-67 | src/utils/templateFileIO.ts:138-145

Por que é explorável: A higienização de PII só é ativada se a flag opcional for passada e analisa unicamente field.defaultValue, ignorando dados pessoais inseridos em títulos de modelos, rótulos (label), textos de apoio ou vetores SVG decorativos em etiquetas personalizadas.

Recomendações Priorizadas de Remediação

- P1 - IMEDIATO

Substituir sanitização SVG regex por DOMPurify / parser estrito: Instalar a biblioteca padrão dompurify com perfil SVG. Aplicar função de escape de entidades HTML em generatePrintableHtml e sanitizar o título em printService.ts.
- P2 - CURTO PRAZO

Ativar Content Security Policy estrito no Tauri e remover unsafe-inline: Configurar política rígida em tauri.conf.json bloqueando inline scripts externos e isolar o iframe de impressão com atributo sandbox="allow-modals".
- P3 - MÉDIO PRAZO

Amadurecer pipeline de privacidade (LGPD): Sanitizar todos os nós textuais do modelo (título, descrição, rótulos e SVGs) ao exportar dados compartilháveis.

Issue 1 para o GitHub

Copie e cole o conteúdo delimitado diretamente no rastreador de issues do repositório:

--- ISSUE 1 ---

[Segurança] Vulnerabilidade de DOM-XSS por evasão do sanitizador SVG via Regex

```
**Labels:** `security`, `severity: high`, `bug`
```

Descrição do Problema

O arquivo ``src/utls/sanitizeSvg.ts`` utiliza expressões regulares simples para higienizar marcações SVG antes da injeção direta no DOM via ``dangerouslySetInnerHTML``. Essa abordagem permite múltiplos bypasses:

1. Tags aninhadas não-recursivas (ex: `<scr<script>ipt>`)
2. Manipuladores de eventos separados por barra sem espaço (ex: `<svg/onload=alert(1)>`)
3. Pseudo-protocolos `javascript:` ofuscados com entidades HTML em atributos `href`.

Evidência no Código

Arquivo: `src/utils/sanitizeSvg.ts:31-45`

```
```typescript
```

```
for (const tag of DANGEROUS_TAGS) {
 const tagRegex = new RegExp(`<${tag}[^>]*>[\\s\\S]*?<\\/${tag}>`, 'gi');
 clean = clean.replace(tagRegex, '');
}
```

```
}
clean = clean.replace(/\s+on[a-z]+=("[']|^")*("[']|^")|(^\\s>+)/gi, '');
}
```

### #### Impacto

Execução arbitrária de código JavaScript no contexto da aplicação desktop/web quando um usuário importa um arquivo vetorial `.svg`` ou modelo `.folium`` manipulado.

#### #### Sugestão de Correção

Adotar uma biblioteca robusta de sanitização baseada em DOM (ex: ``dompurify``) com perfil restrito a SVG:

```
```typescript
```

```
import DOMPurify from 'dompurify';
export const sanitizeSvg = (raw: string) =>
  DOMPurify.sanitize(raw, { USE_PROFILES: { svg: true, svgFilters: true } });
`);
```

Critérios de Aceite

- [] Payload `<svg/onload=alert(1)>` é totalmente desarmado e inofensivo.
- [] Nenhum atributo `on*` ou link com protocolo executável permanece na string higienizada.
- [] Testes unitários cobrindo evasões conhecidas adicionados à suíte de testes.

--- FIM ISSUE 1 ---

Issue 2 para o GitHub

Copie e cole o conteúdo delimitado diretamente no rastreador de issues do repositório:

--- ISSUE 2 ---

```
### [Segurança] XSS armazenado na exportação de templates e interpolação de HTML em doc.write

**Labels:** `security`, `severity: high`

#### Descrição do Problema
Duas rotinas de renderização realizam interpolação direta de variáveis de template em strings HTML sem codificação de entidades (`escapeHtml`):
1. `src/utils/templateFileIO.ts` na função `generatePrintableHtml` (interpola `template.name`, `field.defaultValue` e `template.backgroundSvg` diretamente).
2. `src/utils/printService.ts` na função `executePixelPerfectPrint` (interpola `template.name` na tag `` e grava via `doc.write`).

#### Evidência no Código
`src/utils/templateFileIO.ts:50-76`:
```typescript
const content = field.defaultValue || field.label || '';
return `<div style="..."><span>${content}</span></div>`;
<title>${template.name} - Folium Print</title>
```
`src/utils/printService.ts:73`:
```typescript
<title>Folium Print - ${template.name}</title>
```

#### Impacto
Se um modelo contiver `<script>` no título ou no valor de um campo, a abertura do arquivo exportado ou o envio para o spooler de impressão executará o script com acesso ao contexto do host e ao `localStorage`.

#### Sugestão de Correção
Implementar função de escape HTML antes de qualquer interpolação em marcação estática:
```typescript
const escapeHtml = (str: string) =>
 str.replace(/[<>"]/g, (m) => ({
 '&': '&', '<': '<', '>': '>', '"': '"', "'": '''
 })[m]!));
```

#### Critérios de Aceite
- [ ] Exportação de modelo com caracteres especiais (`<script>`, `&`, `"` ) gera entidades seguras.
- [ ] Iframe de impressão não executa payloads injetados no título ou markup de etiqueta.</pre></div><div data-bbox="88 788 193 800" data-label="Text"><p>--- FIM ISSUE 2 ---</p></div><div data-bbox="62 905 430 920" data-label="Page-Footer">Folium Print — Relatório Confidencial de Auditoria de Segurança</div><div data-bbox="850 905 943 920" data-label="Page-Footer">Página 6 de 8</div>
```

Issue 3 para o GitHub

Copie e cole o conteúdo delimitado diretamente no rastreador de issues do repositório:

--- ISSUE 3 ---

[Segurança] Fortalecimento de Content Security Policy (CSP) no Tauri e Frontend

Labels: `security`, `severity: medium`, `hardening`

Descrição do Problema

O arquivo `src-tauri/tauri.conf.json` desativa a proteção de CSP em nível nativo ao configurar `"csp": null`. Concomitantemente, a tag meta do `index.html` declara `script-src 'self' 'unsafe-inline'`, o que permite a execução direta de qualquer script embutido no DOM, fragilizando a defesa em profundidade da aplicação.

Evidência no Código

```
`src-tauri/tauri.conf.json:22-24`:  
```json  
"security": {
 "csp": null
}
...
`index.html:9`:
```html  
<meta http-equiv="Content-Security-Policy" content="default-src 'self'; script-src 'self'  
'unsafe-inline'; style-src 'self' 'unsafe-inline'; ..." />  
```
```

#### #### Impacto

Na ocorrência de um XSS (como os documentados nas Issues 1 e 2), a ausência de um CSP restritivo permite a execução desimpedida do payload ofensivo.

#### #### Sugestão de Correção

1. Definir uma política rígida no `tauri.conf.json`:

```
```json  
"security": {  
  "csp": "default-src 'self'; script-src 'self'; style-src 'self' 'unsafe-inline'; img-src  
'self' data: blob:;"  
}  
...`
```

2. Migrar manipuladores inline para módulos TypeScript puros e remover `unsafe-inline` da diretiva `script-src`.

Critérios de Aceite

- [] `tauri.conf.json` com CSP estrito ativo sem quebrar os recursos de impressão e visualização.
- [] Scripts injetados em tempo de execução são bloqueados pelo navegador com relatório de violação no console.

--- FIM ISSUE 3 ---

Issue 4 para o GitHub

Copie e cole o conteúdo delimitado diretamente no rastreador de issues do repositório:

--- ISSUE 4 ---

```
### [Segurança] Aprimoramento da higienização de PII em exportações de modelos (LGPD)

**Labels:** `security`, `privacy`, `severity: low`

#### Descrição do Problema
A rotina de higienização de dados pessoais em `src/utils/sanitization.ts` analisa somente o atributo `defaultValue` dos campos do modelo ao exportar dados. Textos inseridos em `template.name`, `template.description`, `field.label` ou nós textuais dentro de vetores SVG não são inspecionados, podendo resultar em vazamento acidental de dados pessoais de clientes ao compartilhar layouts de etiquetas.

#### Evidência no Código
`src/utils/sanitization.ts:49-56`:
```typescript
const sanitizedFields = def.fields.map((field) => {
 if (typeof field.defaultValue === 'string' && detectPotentialPII(field.defaultValue)) {
 return { ...field, defaultValue: maskPIIString(field.defaultValue) };
 }
 return field;
});
```

#### Impacto
Vazamento de dados cadastrais (CPF, e-mails, telefones ou identificadores de pacientes em hospitais) inseridos diretamente na estrutura descritiva do modelo.

#### Sugestão de Correção
Expandir o escopo da função `sanitizeTemplateDefinition` para avaliar recursivamente todos os campos descritivos (`name`, `description`, `label`, `placeholder`) e strings SVG antes da exportação.

#### Critérios de Aceite
- [ ] CPF, e-mail e telefone presentes em `label` ou `description` são mascarados na exportação anônima.
- [ ] Adicionados testes de regressão validando sanitização abrangente de modelos.
```

--- FIM ISSUE 4 ---